



Abschlussprüfung Sommer 2025

Fachinformatiker für Anwendungsentwicklung
Dokumentation zur betrieblichen Projektarbeit

Automatisiertes Testen von Entitäten im Backstage Softwarekatalog

Automatisiertes Validierungssystem, um die Qualität
und Konsistenz der Daten sicherzustellen

eingereicht von:
Maximilian Baron

eingereicht am:
15.05.2025

Ausbildungsbetrieb:
IT-Systemhaus der Bundesagentur für Arbeit
Regensburger Str. 104
90478 Nürnberg

Inhaltsverzeichnis

1. Einleitung	1
1.1 Ausbildungsbetrieb	1
1.2 Ausgangssituation.....	1
1.3 Projektabgrenzung.....	1
2. Projektplanungsphase	2
2.1 IST-Analyse.....	2
2.2 SOLL-Analyse.....	2
2.3 Projektziele festlegen.....	2
2.3.1 Automatisierung	2
2.3.2 Qualitätssicherung	2
2.3.3 Fehlerminimierung	2
2.3.4 Testung des Validierungssystems.....	2
2.4 Zeitplanung	3
2.5 Analyse der Wirtschaftlichkeit.....	3
2.5.1 Einsparpotenzial durch Zeiteffizienz.....	3
2.5.2 Weitere qualitative Vorteile	4
2.5.3 Kosten der Entwicklung	4
2.5.4 Amortisation des Projekts	5
2.5.5 Fazit	5
3. Designphase	6
3.1 Ablauf der Anwendung	6
3.2 Verwendete Technologien und Tools	6
3.3 Bewertung der zu Prüfenden Felder	8
3.4 Systemarchitektur.....	9
3.5 Datenmodellierung und Schnittstellendefinition	9
3.6 Fehlerbehandlungskonzept	9
4. Entwicklungsphase	10
4.1 Umsetzung der Module und Kernfunktionen.....	10
4.2 Implementierung der JSON-Verarbeitung.....	10
4.3 Implementierung des Fehlerhandling und der Berichterstellung	11
4.4 Umgang mit Herausforderungen während der Entwicklung	11
4.5 Angewandte Entwicklungswerkzeuge und -praktiken.....	12
5. Testphase	12
5.1 Planung und Vorbereitung	12
5.2 Durchführung der Tests	12
5.3 Ergebnisse und Optimierung	13

6. Projektabschluss	13
6.1 Projektdokumentation	13
6.2 Übergabe und Präsentation	14
6.3 Soll-Ist-Vergleich	14
6.3.1 Reeller Zeitaufwand	14
6.3.2 Zielanalyse	15
6.4 Fazit	16
6.5 Ausblick	16

Tabellen- und Abbildungsverzeichnis

Tabellen

Tabelle 1: Zeitplanung	3
Tabelle 2: Feldbewertung	8
Tabelle 3: Reeller Zeitaufwand	14
Tabelle 4: Glossar	21

Abbildungen

Abbildung 1: Break-even-Zeitstrahl	5
Abbildung 2: Aktivitätsdiagramm	6
Abbildung 3: Technologie vergleich in Confluence	7
Abbildung 4: Schichtenarchitektur	9
Abbildung 5: Codeausschnitt einer Prüfredel aus <code>internal/validation/validator</code>	10

1. Einleitung

In dieser Dokumentation beschreibe ich die Durchführung meines IHK-Abschlussprojekts mit dem Titel „Automatisiertes Testen von Entitäten im Backstage Softwarekatalog“. Ziel des Projekts ist die Entwicklung eines **Validierungssystems**, das alle Entitäten im **Backstage Softwarekatalog** regelmäßig überprüft und so die Qualität und **Konsistenz** der Daten sicherstellt.

Um die Lesbarkeit und das Verständnis zu verbessern, werden verwendete Fachbegriffe und Abkürzungen in diesem Dokument **fett** markiert und im Glossar am Ende ausführlich erläutert. Methoden-, Klassen-, Bibliotheks- und Feldnamen (z.B. `metadata.name`) werden zur besseren Erkennbarkeit in der Schriftart „**Cascadia Code**“ dargestellt.

1.1 Ausbildungsbetrieb

Die Bundesagentur für Arbeit (**BA**) ist eine Körperschaft des öffentlichen Rechts mit Selbstverwaltungscharakter mit Sitz in Nürnberg. Als der größte Arbeitsmarktdienstleister Deutschlands ist sie unter anderem für die Arbeitsvermittlung, Berufsberatung, Zahlung von Entgeltersatzleistungen und Arbeitsförderungen zuständig.

Die interne Struktur der **BA** gliedert sich in eine Zentrale, Regionaldirektionen und besondere Dienststellen. Mit seinen über 2.000 Mitarbeitern, die auf sieben Standorte verteilt sind, gehört das IT-Systemhaus zu letzteren. Kernaufgaben des IT-Systemhauses sind die Unterstützung der Geschäftsprozesse der **BA** durch die Entwicklung und den Betrieb der IT-Verfahren, die Betreuung der IT-Anwender und das Erstellen von Kommunikationslösungen.

Das IT-Systemhaus ist in die drei wesentlichen Geschäftsbereiche Anwender (A), Kunden (K) sowie zentrale Bereiche und Einheiten unterteilt.

1.2 Ausgangssituation

Mein Abschlussprojekt bearbeite ich im Team **K35** "Technische Plattform"(TP). Dieses Team gehört zur Einheit "35 übergreifende Anwendungen", welche dem Bereich K3 "Geldleistungen 1" zugeordnet ist.

Der Bereich **K3** "Geldleistungen 1" umfasst die Bearbeitung von Leistungen in den Bereichen:

- Arbeitslosengeld
- Familienleistungen
- Arbeitgeberleistungen
- Berufsausbildungsbeihilfe und Rehabilitation
- Übergreifende Anwendungen in Geldleistungen 1

Das Team **K35 TP** stellt Dienstleistungen in den Bereichen Plattform Engineering, Entwickler-Tooling und Anwendungsberatung für den Bereich K3 bereit.

1.3 Projektabgrenzung

Mein Abschlussprojekt umfasst die Konzeption und Entwicklung eines automatisierten **Validierungssystems** für **Entitäten** im **Backstage Softwarekatalog**. Ziel ist es, die **Konsistenz** und Qualität der Daten in diesem Katalog sicherzustellen.

Um den vorgegebenen Zeitrahmen von 80 Stunden nicht zu überschreiten, habe ich bestimmte Funktionserweiterungen bewusst ausgelassen. Dazu zählen beispielsweise:

- die Integration einer Benutzeroberfläche zur Konfiguration neuer Regeln
- direkte Anzeige der Validierungsergebnisse im Backstage-Frontend.

Diese Funktionalitäten können, außerhalb des Abschlussprojekts, zu einem späteren Zeitpunkt ergänzt werden.

2. Projektplanungsphase

2.1 IST-Analyse

Aktuell erfolgt die Validierung von **Entitäten** im **Backstage** Softwarekatalog manuell oder gar nicht. Es existiert keine zentrale Logik zur Prüfung von Pflichtfeldern (z. B. `metadata.name`, `spec.owner`, `spec.type`), Formatvorgaben oder Abhängigkeiten zwischen Entitäten (z. B. `spec.dependsOn`, `spec.consumeApis`). Dies führt zu **inkonsistenten** oder fehlerhaften Einträgen, was andere Teams in ihrer Arbeit beeinträchtigt und den Pflegeaufwand erhöht.

2.2 SOLL-Analyse

Ziel dieses Projekts ist die Entwicklung eines automatisierten **Validierungssystems**, das alle **Entitäten** im **Backstage** Softwarekatalog regelmäßig überprüft. Dadurch soll sichergestellt werden, dass Pflichtfelder korrekt befüllt sind, gültige Werte verwendet werden und Abhängigkeiten zwischen **Entitäten** gültig sind. Zusätzlich sollen die zuständigen Verwalter der **Komponenten** durch automatische E-Mail-Meldungen über fehlerhafte oder **inkonsistente** Daten informiert werden. Die Automatisierung der **Validierung** soll den manuellen Prüfaufwand reduzieren, die Datenqualität im Softwarekatalog verbessern und einen **konsistenten** Standard über alle Teams hinweg sicherstellen.

2.3 Projektziele festlegen

Ich habe die folgenden Ziele festgelegt, um sicherzustellen, dass das **Validierungssystem** den betrieblichen Anforderungen gerecht wird und eine nachhaltige Qualitätsverbesserung der **Backstage**-Daten erreicht wird.

2.3.1 Automatisierung

Ich werde ein **Validierungssystem** entwickeln und implementieren, das in der Lage ist, die Entitäten im **Backstage Softwarekatalog** automatisiert und regelmäßig zu prüfen. Dieses System legt die technische Grundlage für die **Automatisierung** des **Validierungsprozesses**.

2.3.2 Qualitätssicherung

Ich werde die im Abschnitt "3.3. Bewertung zu Prüfender Felder" definierten **Validierungsregeln** für relevante **Entitätsfelder** (inkl. Pflichtfelder, Formate und Abhängigkeiten) technisch umsetzen. Ziel ist die Abdeckung aller spezifizierten Prüfkriterien, um eine fundierte Qualitätssicherung der **Backstage**-Daten zu ermöglichen.

2.3.3 Fehlerminimierung

Ich werde einen Mechanismus zur automatisierten Benachrichtigung der zuständigen Teams über Validierungsfehler implementieren. Dieser Mechanismus versendet strukturierte Fehlerberichte per E-Mail an die jeweiligen Entitätsverwalter(`spec.owner`), um eine schnelle Fehlerkorrektur zu ermöglichen und die Fehlerminimierung zu unterstützen.

2.3.4 Testung des Validierungssystems

Ich werde die korrekte Funktion des entwickelten **Validierungssystems** durch die Konzeption und Durchführung geeigneter Tests (wie Unit-Tests und manuelle Tests mit repräsentativen Datensätzen) umfassend überprüfen. Ziel ist die Identifizierung und Behebung kritischer Fehler vor der Produktivsetzung, um die Zuverlässigkeit des Systems zu gewährleisten. Eine hohe Testabdeckung der Kernvalidierungslogik wird dabei angestrebt.

2.4 Zeitplanung

Die zeitlichen Rahmenbedingungen für dieses Projekt wurden auf 80 Stunden festgelegt. Der nachfolgende Zeitplan zeigt, wie ich die Projektphasen mit deren Arbeitspaketen innerhalb dieses vorgegebenen Zeitrahmens strukturiert und umgesetzt werden.

Projektphase	Arbeitspaket	Dauer in Stunden	
Projektplanungsphase	Erfassung der Anforderungen und Spezifikation	3	Σ10
	Analyse bestehender Prozesse und Herausforderungen	2	
	Analyse der Wirtschaftlichkeit	1	
	Projektziele festlegen	4	
Designphase	Auswahl der Technologien	4	Σ 8
	Erstellung des technischen Konzepts	4	
Implementierungsphase	Implementierung der Schnittstellen	1	Σ 40
	Fehlererkennungs- Meldungssystem umsetzen	26	
	Logging und Monitoring für Validierungsprozesse	13	
Testphase	Manuelle Tests mit verschiedenen Datensätzen	3	Σ 14
	Entwicklung und Durchführung von Unittests	7	
	Optimierung und Fehlerkorrektur	4	
Abschlussphase	Dokumentation	5	Σ 8
	Übergabe	3	
Gesamt		Σ 80	

Tabelle 1: Zeitplanung

2.5 Analyse der Wirtschaftlichkeit

Die Entwicklung des automatisierten **Validierungssystems** für den **Backstage Softwarekatalog** bietet erhebliche wirtschaftliche Vorteile für das IT-Systemhaus der Bundesagentur für Arbeit. Durch die Automatisierung der Validierungsprozesse wird nicht nur der manuelle Prüfaufwand reduziert, sondern auch die Qualität der Daten im **Softwarekatalog** verbessert, was sich in einer gesteigerten Effizienz der Entwicklungsteams niederschlägt.

2.5.1 Einsparpotenzial durch Zeiteffizienz

Momentan verbringen Entwickler einen erheblichen Teil ihrer Arbeitszeit mit der Suche und dem Zusammentragen von Informationen über **Softwarekomponenten**. Dies umfasst Tätigkeiten wie die Suche nach Schnittstellen, **API**-Dokumentationen, Zuständigkeiten und anderen relevanten Informationen, die für die Entwicklung und Wartung von Softwarekomponenten benötigt werden. Aufgrund der dezentralen Dokumentation und der unzureichenden Konsistenz der Daten im Softwarekatalog ergibt sich ein hoher Suchaufwand.

Eine interne Schätzung von dem Team **K35** ergab, dass Entwickler durchschnittlich etwa 4 Stunden pro Woche mit diesen Suchaufgaben beschäftigt sind. Das automatisierte **Validierungssystem** und die damit verbundene Verbesserung der Datenqualität im **Softwarekatalog** werden den Suchaufwand voraussichtlich um 80 % reduzieren. Da alle relevanten Informationen einheitlich und vollständig vorliegen, entfällt die zeitaufwendige Suche in verschiedenen Systemen und die manuelle Überprüfung der Daten.

Im IT-Systemhaus der **BA** arbeiten durchschnittlich 120 Entwickler mit dem **Backstage Softwarekatalog**. Bei einer wöchentlichen Einsparung von 3,2 Stunden pro Entwickler (= 4 Stunden * 0,8) ergibt sich eine wöchentliche Gesamteinsparung von 384 Stunden (= 120 Entwickler * 3,2 Stunden/Entwickler). Hochgerechnet auf einen Monat ($\frac{384 \text{ Stunden/Woche} * 52 \text{ Wochen}}{12 \text{ Monate}}$) entspricht dies einer Einsparung von 1.664 Stunden.

Um die monetäre Auswirkung dieser Zeitersparnis zu bewerten, muss ein Stundensatz für die Entwickler angesetzt werden. Der durchschnittliche Bruttomonatslohn eines Entwicklers im IT-Systemhaus der BA beträgt 5.500 Euro. Bei einer wöchentlichen Arbeitszeit von 39 Stunden ergibt sich ein Stundensatz von 32,54 Euro ($= \frac{5.500 \text{ €} * 12 \text{ Monate}}{52 \text{ Wochen} * 39 \text{ Stunden}}$). Die monatliche Gesamteinsparung beträgt somit 54.146,56 Euro ($= 1.664 \text{ Stunden} * 32,54 \text{ Euro/Stunde}$).

2.5.2 Weitere qualitative Vorteile

Neben der direkten Zeitersparnis ergeben sich weitere qualitative Vorteile durch das automatisierte **Validierungssystem**: Die Automatisierung reduziert die Anzahl von Fehlern und **Inkonsistenzen** im **Softwarekatalog**, was die Zuverlässigkeit und Nutzbarkeit der Daten erhöht. Neue Mitarbeiter können sich schneller in die Softwarelandschaft einarbeiten, da alle Informationen konsistent und vollständig dokumentiert sind. Entwickler sparen Zeit bei der Suche nach relevanten Informationen, da die Daten im **Softwarekatalog** zuverlässiger sind. Projektmanager profitieren von einer besseren Planbarkeit, da Abhängigkeiten zwischen Komponenten klarer ersichtlich sind. Die automatisierte **Validierung** hilft, Fehler frühzeitig zu erkennen und zu vermeiden, was den Aufwand für die Fehlerbehebung reduziert.

Darüber hinaus ist zu erwarten, dass die verbesserte Datenqualität und die dadurch gesteigerte Zuverlässigkeit des **Backstage Softwarekatalogs** zu einer Erhöhung der Benutzeranzahl führen werden. Aktuell nutzen durchschnittlich 120 Entwickler den Katalog regelmäßig. Wenn sich die Benutzeranzahl beispielsweise um 20 % erhöht, bedeutet dies, dass zukünftig 24 zusätzliche Entwickler von der Zeitersparnis profitieren. Dies würde das Gesamteinsparpotenzial des Systems noch weiter steigern.

2.5.3 Kosten der Entwicklung

Für die Umsetzung des Projekts wurde ein Gesamtaufwand von 80 Stunden veranschlagt. Zur Kalkulation der Entwicklungskosten wird der jeweilige Stundensatz zugrunde gelegt, der sich aus dem monatlichen Bruttolohn sowie der wöchentlichen Arbeitszeit ableitet - sowohl für mich als Auszubildenden als auch für den eingebundenen Stakeholder.

Mein monatlicher Bruttolohn beträgt 1.414,02 Euro bei einer vertraglich festgelegten wöchentlichen Arbeitszeit von 39 Stunden. Daraus ergibt sich ein Stundensatz von 8,36 Euro ($= \frac{1.414,02 \text{ €} * 12 \text{ Monate}}{52 \text{ Wochen} * 39 \text{ Stunden}}$). Multipliziert mit den geplanten 80 Stunden Arbeitsaufwand ergibt sich ein Kostenanteil in Höhe von 668,80 Euro ($= 8,36 \text{ €} * 80 \text{ Stunden}$).

Darüber hinaus war auch mein betriebspraktischer Unterweiser (**BPU**) in das Projekt eingebunden. Für seine Unterstützung, insbesondere während der Planungsphase, wird ein Zeitaufwand von 4 Stunden angesetzt. Sein monatlicher Bruttolohn beträgt gemäß Tarifvertrag der **BA (TV-BA¹, Tätigkeitsebene 2, Tarifstufe 3)** 5.483,11 Euro, was einem Stundensatz von 32,45 Euro ($= \frac{5.483,11 \text{ €} * 12 \text{ Monate}}{52 \text{ Wochen} * 39 \text{ Stunden}}$) entspricht. Daraus ergeben sich zusätzliche Kosten von 129,76 Euro ($= 32,45 \text{ €} * 4 \text{ Stunden}$).

Die Gesamtkosten der Projektentwicklung setzen sich aus meinem eigenen Aufwand in Höhe von 668,80 Euro sowie dem zeitlichen Aufwand des Stakeholders mit 129,76 Euro zusammen. Damit belaufen sich die gesamten Entwicklungskosten auf 798,56 Euro.

2.5.4 Amortisation des Projekts

Die Amortisationszeit ist der Zeitraum, in dem die Einsparungen die Entwicklungskosten übersteigen. Die monatliche Einsparung durch die Reduktion des Suchaufwands beträgt 54.146,56 Euro. Die wöchentliche Einsparung beträgt 12.495,36 Euro ($= \frac{54.146,56 \text{ €} * 12 \text{ Monate}}{52 \text{ Wochen}}$). Theoretisch gesehen würde sich das Projekt bereits nach 0,06 Wochen (ca. ½ Tag) amortisieren

$$\left(= \frac{\text{Entwicklungskosten}}{\text{Einsparung € /Woche}} = \frac{798,56 \text{ €}}{12.495,36 \text{ € /Woche}} \right).$$

In der Praxis ist jedoch zu berücksichtigen, dass die Teams Zeit benötigen, um die durch das **Validierungssystem** identifizierten Fehler und **Inkonsistenzen** in den **Entitäten** zu beheben. Es ist realistisch anzunehmen, dass dieser Prozess etwa eine Woche in Anspruch nehmen wird. Daher ist mit einer tatsächlichen Amortisationszeit von ca. einer Woche zu rechnen.

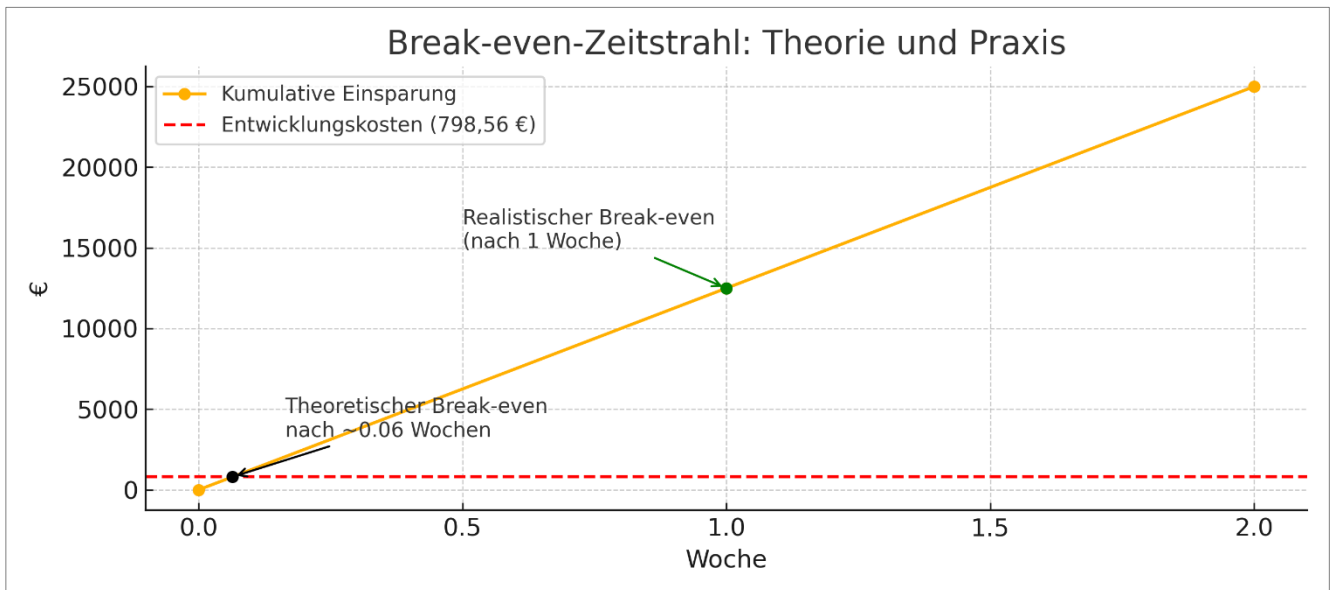


Abbildung 1: Break-even-Zeitstrahl

2.5.5 Fazit

Die Analyse der Wirtschaftlichkeit zeigt, dass die Entwicklung des automatisierten **Validierungssystems** eine äußerst lohnende Investition ist. Die Einsparungen durch die gesteigerte Effizienz und die verbesserte Datenqualität übersteigen die Entwicklungskosten bereits nach sehr kurzer Zeit. Darüber hinaus ergeben sich zahlreiche qualitative Vorteile, die die langfristige Produktivität und Wettbewerbsfähigkeit des IT-Systemhauses positiv beeinflussen. Die zu erwartende Steigerung der Benutzeranzahl des **Backstage Softwarekatalogs** durch die verbesserte Datenqualität unterstreicht das hohe wirtschaftliche Potenzial des Projekts noch weiter.

3. Designphase

In der Designphase habe ich das Grobkonzept aus der Planung in ein technisches Design überführt. Ziel war es, eine wartbare und erweiterbare Architektur zu schaffen, die eine effiziente Umsetzung und spätere Anpassbarkeit ermöglicht.

3.1 Ablauf der Anwendung

Den Ablauf des **Validierungssystems** habe ich durch ein Aktivitätsdiagramm veranschaulicht (siehe Abbildung 2). Dieses Diagramm zeigt die Interaktionen zwischen dem System und dem **Backstage Softwarekatalog** sowie die internen Abläufe innerhalb des **Validierungssystems**.

Der **Validierungsprozess** beginnt mit einem automatischen Start durch einen Zeitplan. Das System ruft dann alle **Entitäten** aus dem **Backstage Softwarekatalog** über die **Backstage API** ab. Für jede abgerufene **Entität** werden systematisch die definierten Validierungsregeln angewendet, um zu prüfen, ob die **Entität** vollständig und korrekt ist (z.B. ob alle Pflichtfelder vorhanden sind, ob Datenformate korrekt sind). Wenn bei der **Validierung** Fehler gefunden werden, wird ein strukturierter Fehlerbericht erstellt, der die Details der Fehler enthält. Dieser Fehlerbericht wird per E-Mail an das verantwortliche Team gesendet. Danach oder wenn es keine Fehler gab, wird geprüft, ob noch nicht geprüfte **Entitäten** vorhanden sind. Sind noch nicht geprüfte Entitäten vorhanden, wird der Prozess fortgesetzt. Nachdem alle **Entitäten** geprüft wurden, wird ein gesamter Fehlerbericht erstellt und der Prozess ist zu Ende.

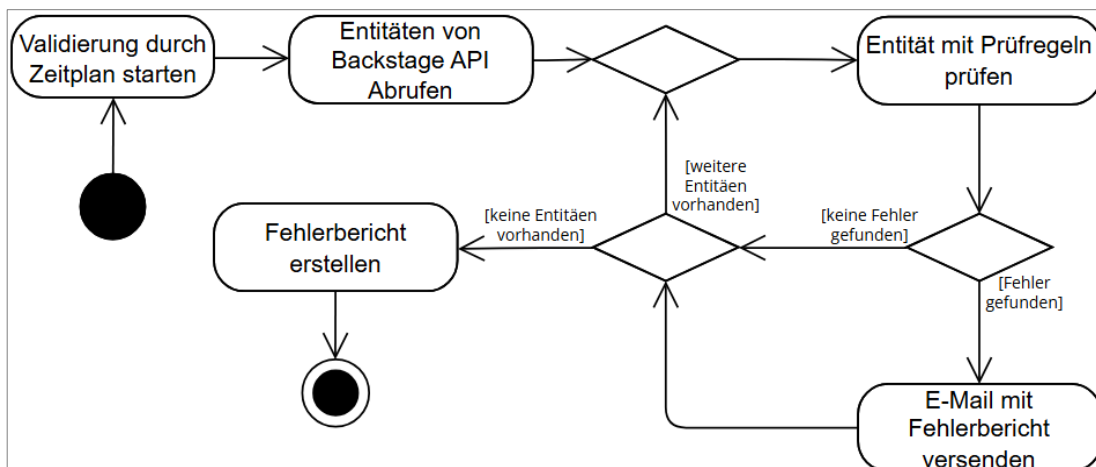


Abbildung 2: Aktivitätsdiagramm

3.2 Verwendete Technologien und Tools

Für die Umsetzung des **Validierungssystems** wurden von mir gezielt Technologien und Tools ausgewählt, die eine effiziente Entwicklung sowie eine einfache Wartung und Erweiterbarkeit ermöglichen. Im Vorfeld habe ich verschiedene Optionen intensiv geprüft und verglichen. Dabei habe ich besonders auf die Kompatibilität mit der bestehenden Infrastruktur sowie auf die Eignung für Backend-nahe Aufgaben geachtet.

Ich habe mich für die Programmiersprache **Go** entschieden, da sie durch ihre Effizienz und klare Syntax ideal für automatisierte Backend-Prozesse ist. Im Vergleich zu Java oder Node.js bietet **Go** weniger Umfang und dennoch genug, was zu einer deutlich besseren Performance bei paralleler Verarbeitung beiträgt.

Als Entwicklungsumgebung kam **IntelliJ IDEA Ultimate** zum Einsatz, da sie eine hervorragende Integration für **Go** bietet und im Betrieb bereits lizenziert vorhanden ist. Alternativen wie VS Code wurden zwar geprüft, boten jedoch nicht denselben Funktionsumfang im Bereich **Debugging** und Codeanalyse.

Für die **Versionsverwaltung** habe ich **Git** genutzt. Es ist branchenweit etabliert, ermöglicht paralleles Arbeiten im Team und ist durch seine Verbreitung gut dokumentiert. Systeme wie SVN wurden aufgrund geringerer Flexibilität verworfen.

Das Aufgabenmanagement sowie die Dokumentation erfolgten über **Jira** und **Confluence**, da diese bereits fester Bestandteil des Entwicklungsprozesses im Betrieb sind. Im Vergleich zu Tools wie Trello oder Notion bieten sie tiefere Integrationen in bestehende Workflows und erlauben eine strukturierte Nachverfolgbarkeit.

Für die Visualisierung und grafische Darstellung der Systemzusammenhänge benutze ich **visual-paradigm**² und **excalidraw**³.

Die Kommunikation mit dem **Softwarekatalog** erfolgt über die **Backstage API**. Diese Schnittstelle ermöglicht es, die Entitäten zentral auszulesen, ohne direkten Datenbankzugriff zu benötigen – ein klarer Vorteil gegenüber individuell entwickelten REST-Endpunkten, die oft weniger wartungsfreundlich und nicht systemübergreifend einsetzbar sind.

Für die E-Mail-Benachrichtigung wurde ein integrierter **SMTP-Client** verwendet. Dieser erlaubt einen direkten Versand der Fehlermeldungen an die zuständigen Teams. Komplexere Messaging-Lösungen wie Message Queues (z. B. RabbitMQ) wurden aufgrund des einfachen Anwendungsfalls nicht benötigt.

Zur Umsetzung nutzte ich vorwiegend Standard**bibliotheken** wie `net/http`, `encoding/json`, `net/smtp`, `log`, `regexp` und `time`, da sie robust, gut dokumentiert und für meine Anforderungen vollkommen ausreichend sind. Für die **Unit-Tests** kam zusätzlich das externe Paket `testify` zum Einsatz, das im Vergleich zu nativen Testfunktionen eine komfortablere **Assertion-Logik** bietet.

Nachfolgend ist eine Abbildung aus dem internen **Confluence-Wiki** welches ein Auszug von der Planung zur Auswahl der Technologien dargestellt:

2. Programmiersprache:		
Sprache	Vorteile	Nachteile
Go	• Schnell • Minimalistisch • Parallelisierbar	• Begrenztes Ökosystem im Vergleich zu Java
Java	• Gut für komplexe und skalierbare Projekte	• Schwergewichtiger • mehr Boilerplate-Code
Node.js	• Schnell für Prototypen und Webprojekte	• Weniger geeignet für CPU intensive Aufgaben

Entscheidung:

Go wurde gewählt, da es schnell, effizient und ideal für Netzwerkoperationen wie REST-API-Aufrufe ist. Zudem ermöglicht die einfache Syntax einen schnellen Einstieg.

Abbildung 3: Technologievergleich in Confluence

3.3 Bewertung der zu Prüfenden Felder

Die Auswahl und Priorisierung der Validierungsfelder erfolgte in enger Anlehnung an die tatsächlichen Anforderungen an den **Softwarekatalogs**. Die Bewertung orientiert sich an ihrer Bedeutung für die Funktionalität und Nachvollziehbarkeit der Komponenten.

Die Tabelle in diesem Abschnitt dokumentiert die Regeln, Formate und die Wichtigkeiten der einzelnen Felder.

Feldname	Erläuterung	Prüfregeln	Wichtigkeit
<code>metadata.name</code>	Eindeutiger Name der Entität	keine Sonderzeichen/Leerzeichen	hoch
<code>spec.owner</code>	Verantwortliches Team oder Person	Muss existierendes Team/Nutzer sein	hoch
<code>kind</code>	Entitätstyp	Müssen aus gültiger Liste stammen (<code>component</code> , <code>API</code>)	hoch
<code>apiVersion</code>	API-Version des Manifests	Format <code>group/version</code> z.B. <code>Backstage.io/v1alpha</code>	hoch
<code>spec.type</code>	Art der Entität	muss aus vordefinierter Liste stammen	hoch
<code>spec.lifecycle</code>	Status der Entität (z. B. <code>experimental</code> , <code>production</code>)	muss aus Liste (<code>experimental</code> , <code>production</code> , <code>deprecated</code>) stammen	mittel
<code>meta-data.description</code>	Beschreibung der Entität	Mindestend 10 Zeichen, keine reinen Platzhaltertexte	mittel
<code>metadata.tags</code>	Gruppierung von Entitäten	Namespace muss existieren	mittel
<code>metadata.annotations</code>	Zusätzliche Metadaten und Labels	Format muss stimmen (<code>key: value</code>)	mittel
<code>spec.system</code>	Zugehöriges System	System muss existieren	mittel
<code>spec.dependsOn</code>	Abhängigkeiten zu anderen Entitäten	Abhängigkeit muss gültig sein	mittel
<code>spec.provideApis</code>	APIs, die diese Entität bereitstellt	API muss existieren	mittel
<code>spec.consumeAPI</code>	APIs, die von dieser Entität genutzt werden	API muss existieren	mittel
<code>spec.subcomponentOf</code>	Übergeordnete Entität zu, der sie gehört	Entität muss Existieren	mittel

Tabelle 2: Feldbewertung

Die dargestellten Felder bilden die Grundlage für eine umfassende und automatisierbare Qualitätsprüfung im **Backstage Softwarekatalog**. Durch die Priorisierung nach Wichtigkeit konnte ich sicherstellen, dass vor allem kritische Informationen wie Verantwortlichkeiten, Typen und Namen der Entitäten zuverlässig **validiert** werden. Mittelfristig können zusätzliche Felder integriert werden, um die Tiefe der Prüfung weiter zu erhöhen – insbesondere im Bereich der Beziehungsfelder oder optionalen **Metadaten**.

3.4 Systemarchitektur

Das **Validierungssystem** habe ich auf Basis einer **modularen Schichtenarchitektur** umgesetzt, um eine klare Trennung von Verantwortlichkeiten, hohe Wartbarkeit und einfache Erweiterbarkeit sicherzustellen. Die Anwendung ist als eigenständiger **Microservice** realisiert und vollständig im Backend angesiedelt.

Der Einstiegspunkt der Anwendung befindet sich in der Datei `cmd/microservice/main.go`. Von dort aus wird die gesamte Anwendungslogik gesteuert. Die interne Logik ist im Verzeichnis `internal` gekapselt, das in mehrere logisch getrennte Module unterteilt ist:

- **validation**
Dieses Modul enthält sämtliche Validierungsregeln. Hier wird geprüft, ob Pflichtfelder vorhanden sind, Formatvorgaben eingehalten werden und ob verknüpfte **Entitäten** gültig sind. Die **Validierung** erfolgt regelbasiert und ist leicht erweiterbar.
- **models**
In diesem Modul sind die Datenstrukturen definiert, mit denen die **Entitäten** verarbeitet werden. Die Struktur ist eng an die **Backstage-YAML-Spezifikation** angelehnt, um eine verlustfreie Abbildung der **API-Daten** zu gewährleisten.
- **backstage**
Dieses Modul übernimmt die Kommunikation mit dem **Backstage Softwarekatalog**. Über **HTTP-Requests** werden die **Entitäten** geladen, **paginiert** verarbeitet und an die `validation` übergeben.
- **mailer**
Dieses Modul ist für den **SMTP-basierten** Versand von Fehlerberichten zuständig. Für jede fehlerhafte **Entität** wird ein individueller Bericht erstellt und automatisiert an das verantwortliche Team (basierend auf `spec.owner`) versendet. Die zentrale Funktion `SendErrorReport()` kapselt die Logik des E-Mail-Versands.

Diese klare Trennung der Aufgabenbereiche ermöglicht eine gezielte Weiterentwicklung einzelner Komponenten, z.B. bei der Integration neuer Prüfregeln, alternativer Benachrichtigungskanäle oder einer Datenpersistenz. Die modulare Architektur wurde bewusst so konzipiert, dass zukünftige Erweiterungen – wie etwa eine grafische Benutzeroberfläche oder eine systemweite Fehlerhistorie – problemlos integrierbar sind.

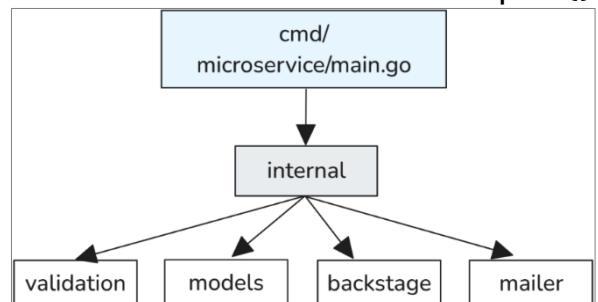


Abbildung 4: Schichtenarchitektur

3.5 Datenmodellierung und Schnittstellendefinition

Die Verarbeitung der Entitäten erfolgt in **Go** mittels speziell entwickelter **Structs**, die exakt auf die Struktur der von der **Backstage API** gelieferten **JSON-Entitäten** abgestimmt sind, welche wiederum auf den **YAML-Definitionen** im Katalog basieren. Diese befinden sich im Modul `models` und erlauben durch die Nutzung von `encoding/json` eine verlustfreie Serialisierung. Die Übergabe zwischen `backstage`, `validation` und `mailer` erfolgt über strukturierte **Go-Objekte**, wodurch eine lose Kopplung und hohe Testbarkeit gewährleistet wird.

3.6 Fehlerbehandlungskonzept

Das Fehlerbehandlungskonzept beschreibt den Aufbau eines **modularen Systems** zur Erkennung, Kategorisierung und Weiterleitung von Validierungsfehlern. Jede Abweichung wird strukturiert erfasst und durch die Funktion `GenerateErrorReport()` zu einem Bericht zusammengefasst. Die Versendung erfolgt über das Modul `mailer`. Die technische Umsetzung dieser Konzepte wird in Abschnitt 4.3 detailliert beschrieben.

4. Entwicklungsphase

In der Entwicklungsphase habe ich das technische Konzept aus der Designphase Schritt für Schritt in funktionierenden und testbaren Code überführt. Mein Ziel war es, ein robustes, wartbares und erweiterbares **Validierungssystem** zu realisieren. Das System überprüft automatisiert alle **Entitäten** des **Backstage Softwarekatalogs** im Backend. Die Umsetzung erfolgte in der Programmiersprache **Go**, wobei ich den Einstiegspunkt der Anwendung in der Datei `cmd/microservice/main.go` definiert habe. Von dort aus steuere ich die Hauptmodule: **backstage** (API-Zugriff), **validation** (Prüflogik), **mailer** (Benachrichtigungen) und **models** (Datenstrukturen).

Ich habe die einzelnen Komponenten **iterativ** entwickelt und kontinuierlich durch manuelle Tests und **Unit-Tests** überprüft, um eine stabile Basis sicherzustellen.

4.1 Umsetzung der Module und Kernfunktionen

Die Implementierung erfolgte schrittweise, jeweils modulbasiert.

- **backstage**-Modul: Mit der Funktion `FetchAllEntities()` werden paginierte **API-Responses** verarbeitet. Die Herausforderung bestand in der dynamischen **Pagination**. Es kamen **net/http** und eine einfache **Retry-Logik** zum Einsatz
- **modules**-Modul: Die **Structs** wurden anhand realer **YAML-Dateien** erstellt. Probleme traten bei optionalen Feldern auf, die durch gezielte **omitempty-Annotationen** gelöst wurden.
- **validation**-Modul: Die zentrale Funktion `ValidateEntities()` implementiert alle Prüfre-geln.

Beispiel einer Prüfung (`metadata.name`):

```
if entity.Metadata.Name == "" {  
    errorCountPerRule["metadata.name fehlt"]++  
    log.Printf("Entität %s: metadata.name fehlt", entity.Metadata.Name)  
} else if matched, _ := regexp.MatchString("^[a-z0-9\\-]+$", entity.Me-  
tadata.Name); !matched {  
    errorCountPerRule["metadata.name ungültig"]++  
    log.Printf("Entität %s: metadata.name enthält ungültige Zeichen",  
entity.Metadata.Name)  
}
```

Abbildung 5: Codeausschnitt einer Prüfregele aus `internal/validation/validator`

- **mailer**-Modul: Die Funktion `SendErrorReport()` erstellt eine **SMTP-Mail** mit Fehlerdetails. Der Betreff enthält die betroffene Komponente, der Inhalt listet alle Fehler sortiert nach Regel-typ auf. Der Versand erfolgt über eine einfache **SMTP-Session** mit **net/smtp**.

4.2 Implementierung der JSON-Verarbeitung

Die von der **Backstage API** gelieferten **JSON-Daten** wurden mit `encoding/json` verarbeitet und in **Structs** gemappt. Besonders beachtet wurden dabei

- Verschachtelte Felder wie `spec.dependencies`
- Optionalität von Feldern mit **Pointern**
- Konsistent zur **YAML-Definition** im Katalog

So wurde sichergestellt, dass auch bei Änderungen in der **API** eine schnelle Anpassung möglich ist.

In der folgenden Abbildung 5 ist ein Codeausschnitt von solch einem **Struct**.

```
type EntityItem struct {
    Kind      string `json:"kind"`
    ApiVersion string `json:"apiVersion"`
    Metadata  struct {
        Name      string `json:"name"`
        Description string `json:"description"`
        Annotations map[string]string `json:"annotations"`
        Tags      []string `json:"tags"`
    } `json:"Metadata"`
}
```

Abbildung 5: Codeausschnitt von einem **Struct** aus `internal/models/entity`

4.3 Implementierung des Fehlerhandling und der Berichterstellung

Ich habe das Fehlerhandling so implementiert, dass das System auf Basis definierter Validierungsregeln Fehler erkennt. Jede Abweichung wird kategorisiert und mit einer aussagekräftigen Textnachricht dokumentiert, die Angaben zur betroffenen Regel enthält.

Beispiel:

- Regelname (z. B. `metadata.name`)
- Betroffene Entität (ID)
- Beschreibung (konkrete Ursache)

Diese Fehlermeldungen werden als strukturierte Einträge gesammelt und für die weitere Verarbeitung genutzt. Die Funktion `GenerateErrorReport()` fasst alle für diese **Entität** erkannten Regelverstöße in einem strukturierten Textbericht zusammen. Der Versand dieses Berichts erfolgt automatisiert über das Modul `mailer`. So kommt es als eine formatierte E-Mail direkt an das zuständige Team an. Die Empfängeradresse wird dabei aus dem Feld `spec.owner` ermittelt. So erhält jedes Team nur die Informationen zu den Entitäten, für die es verantwortlich ist.

4.4 Umgang mit Herausforderungen während der Entwicklung

Während der Entwicklung bin ich auf mehrere Herausforderungen gestoßen:

- **API-Kompatibilität mit Backstage:** Da die **Backstage-Instanz** im IT-Systemhaus selbst gehostet wird, basiert sie nicht auf der neuesten Version. Dadurch kam es anfangs zu Problemen mit der **API-Dokumentation** und der tatsächlichen **API-Response**. Ich habe die Unterschiede manuell analysiert und meinen Code entsprechend angepasst.
- **Pagination bei der Backstage API:** Die **API** liefert die **Entitäten** seitenweise aus. Ich habe eine Schleife mit automatischer Weiterverfolgung und Fehlerbehandlung entwickelt, um alle Daten zuverlässig einzulesen.
- **Umgang mit optionalen Feldern:** Einige Felder sind optional. Um Fehler beim **Unmarshalling** zu vermeiden, habe ich **Pointer-Felder** verwendet und mit **omitempty-Tags** gearbeitet.
- **Formatierung der E-Mail:** Die übersichtliche Darstellung von Fehlermeldungen in E-Mails stellte sich als schwierig heraus. Ich habe eine gruppierte Darstellung mit Sortierung nach Fehlerart und betroffener Entität entwickelt.

4.5 Angewandte Entwicklungswerkzeuge und -praktiken

Da ich die verwendeten Technologien bereits in Abschnitt 3.2 ausführlich begründet habe, liste ich hier nur auf, wie ich sie konkret in der Entwicklungsphase genutzt habe:

Ich habe **Git** zur strukturierten Quellcodeverwaltung eingesetzt. **IntelliJ IDEA Ultimate** unterstützte mich beim **Debugging** und bei der Entwicklung mit Code-Inspektion. Die technische Dokumentation und Aufgabenverfolgung führte ich in **Confluence**.

5. Testphase

5.1 Planung und Vorbereitung

Für die Qualitätssicherung meines Validierungssystems plante ich umfassende Tests in zwei zentralen Bereichen:

- dem Modul `validation`, welches die Kernlogik der Entitätsprüfung enthält,
- sowie den Modul `mailer`, das für die Generierung und den der Fehlerberichte zuständig ist.

Ziel war es, eine möglichst hohe Testabdeckung zu erreichen, insbesondere für alle relevanten Prüflogikpfade und potenzielle Fehlerquellen. Um dabei unabhängig von externen Systemen (wie z. B. der **Backstage API** oder einem **SMTP-Server**) testen zu können, setzte ich gezielt auf den Einsatz von **Mock-Objekten**. Dadurch konnte ich realitätsnahe Tests durchführen, ohne auf Live-Daten oder Netzwerkinfrastruktur angewiesen zu sein.

Zusätzlich definierte ich typische Fehlerkonstellationen in **YAML-Dateien**, um auch **End-to-End-Szenarien** innerhalb der späteren Integrationsumgebung zu testen.

5.2 Durchführung der Tests

Ich erstellte Unit-Tests für jede einzelne Validierungsregel im Modul `validation`, z.B. für:

- fehlende Pflichtfelder (`metadata.name`),
- ungültige Werte (z.B. leere Lifecycle-Strings),
- sowie fehlerhafte Referenzen auf abhängige **Entitäten** (`spec.dependsOn`).

Dabei überprüfe ich nicht nur die korrekte Fehlererkennung, sondern auch die strukturierte Ausgabe des Fehlerobjekts, das später im Fehler verwendet wird.

Im Modul `mailer` simulierte ich die Generierung und den Versandprozess mittels eines Dummy-**SMTP-Clients**. Ich prüfe unter anderem:

- ob die Mail korrekt formatiert ist,
- ob der korrekte Empfänger aus dem Feld `spec.owner` ermittelt wurde,
- und ob alle validierten Fehler übersichtlich dargestellt wurden.

Zusätzlich führte ich mehrere **End-to-End-Tests** durch, bei denen ich realistische **YAML-Entitäten** einspielte und den gesamten Ablauf - von der **Entität** bis zur fertigen E-Mail - automatisiert durchlief.

5.3 Ergebnisse und Optimierung

Die Tests deckten eine Vielzahl an realistischen Fehlerfällen ab und ermöglichten es mir, kritische Schwachstellen frühzeitig zu identifizieren.

Ein konkretes Beispiel war die Prüfung des Feldes `spec.owner`. Hierbei trat ein Fehler auf, wenn der Wert explizit auf `null` gesetzt war - in **Go** führte dies zu einem unerwarteten Verhalten. Ich überarbeitete daraufhin die betroffene Logik und ergänzte eine explizite Prüfung auf `nil`, um diese Sonderfälle korrekt zu behandeln.

Ein weiteres Problem ergab sich beim Zusammenspiel mehrerer Validierungsregeln in verschachtelten Strukturen - insbesondere bei verschachtelten Definitionen und Text-Feldern. Auch diese wurden entsprechend abgesichert.

Nach der Optimierung konnte ich eine vollständige Abdeckung aller Kernfunktionen sicherstellen. Die **Unit-Tests** wurden automatisiert über `go test ./...` ausgeführt und dokumentiert. Durch die konsequente Trennung von Validierungslogik und Seiteneffekten (wie dem Mailversand) erwies sich das System als modular testbar und leicht erweiterbar.

6. Projektabschluss

Nachdem die Entwicklungs- und Testphasen erfolgreich abgeschlossen wurden, widmete ich mich dem formellen Projektabschluss. In diesem Kapitel beschreibe ich die Erstellung der Projektdokumentation selbst, die Übergabe der entwickelten Lösung an mein Team sowie die Präsentation meiner Arbeitsergebnisse. Weiterhin ziehe ich einen Soll-Ist-Vergleich der Zeitplanung und der definierten Projektziele, formuliere ein Fazit zum Gesamtprojekt und gebe einen Ausblick

6.1 Projektdokumentation

Die vorliegende Dokumentation ist das zentrale Artefakt des Projektabschlusses. Ich habe sie parallel zur Projektarbeit erstellt und fortlaufend aktualisiert. Ihr Ziel ist es, den gesamten Prozess von der Projektplanung über das Design, die Entwicklung und die Testphase detailliert und nachvollziehbar abzubilden.

Die Dokumentation richtet sich primär an die IHK-Prüfer, dient aber auch als wichtige interne Ressource für das Team **K35 (TP)** im IT-Systemhaus. Sie enthält alle relevanten Informationen zur Konzeption, Implementierung und Nutzung des automatisierten **Validierungssystems**. Dazu gehören unter anderem:

- Eine detaillierte Beschreibung der Ausgangssituation und des Projektziels.
- Die Analyse der Anforderungen und die Definition der zu prüfenden **Entitätsfelder**.
- Die getroffenen Architektur- und Technologieentscheidungen mit Begründung.
- Die technische Umsetzung der einzelnen Module und Kernfunktionen in **Go**.
- Das Konzept und die Implementierung des Fehlerhandlings und der Berichterstattung per E-Mail.
- Die Planung und Durchführung der Testphase, einschließlich der verwendeten Testfälle und der Ergebnisse.
- Eine Analyse der Wirtschaftlichkeit des Projekts.
- Ein Glossar zur Erläuterung relevanter Fachbegriffe und Abkürzungen.

Ich habe bei der Erstellung großen Wert auf Formalia wie eine klare Struktur, einheitliche Formatierung und die Einhaltung der vorgegebenen Seitenanzahl gelegt. Die Dokumentation ist das schriftliche Zeugnis meiner im Rahmen des Abschlussprüfungsprojekts erbrachten Leistungen.

6.2 Übergabe und Präsentation

Nachdem die Entwicklung und grundlegenden Tests des Validierungssystems abgeschlossen waren, habe ich die Anwendung und die zugehörige Dokumentation formal an mein Team **K35 (TP)** übergeben. Die Übergabe umfasste:

- Die Bereitstellung des vollständigen Quellcodes im unternehmensinternen Versionsverwaltungssystem (**Git**).
- Eine technische Einführung in die Architektur und den Aufbau des **Validierungssystems** für die Teammitglieder.
- Die Dokumentation der Deployment-Schritte und der notwendigen Konfiguration für den Betrieb in der Zielumgebung.
- Die Übergabe der erstellten Dokumentation als Nachschlagewerk.

6.3 Soll-Ist-Vergleich

6.3.1 Reeller Zeitaufwand

Im Rahmen des **Projektabschlusses** habe ich einen detaillierten Vergleich zwischen der geplanten **Zeitplanung** im **Projektantrag** und den tatsächlich aufgewendeten Zeiten durchgeführt. Die im **Projektantrag** veranschlagte Gesamtzeit von 80 Stunden konnte punktgenau eingehalten werden. Die folgende Tabelle dokumentiert den geplanten (SOLL) und den tatsächlichen (IST) Zeitaufwand sowie die daraus resultierenden Abweichungen für die einzelnen Arbeitspakete und Projektphasen:

Projektphase	Arbeitspakete	SOLL-Dauer in Stunden		Abweichung in Stunden	
Projektplanungs- phase	Erfassung der Anforderungen und Spezifikation	3	Σ 10	Δ +1	Δ +1
	Analyse bestehender Prozesse und Herausforderungen	2		Δ 0	
	Analyse der Wirtschaftlichkeit	1		Δ 0	
	Projektziele festlegen	4		Δ 0	
Designphase	Auswahl der Technologien	4	Σ 8	Δ +1	Δ +2
	Erstellung des technischen Konzepts	4		Δ +1	
Implementierungs- phase	Implementierung der Schnittstellen	1	Σ 40	Δ 0	Δ -5
	Fehlererkennungs- Meldungssystem umsetzen	26		Δ -3	
	Logging und Monitoring für Validierungsprozesse	13		Δ -2	
Testphase	Manuelle Tests mit verschiedenen Datensätzen	3	Σ 14	Δ +1	Δ +2
	Entwicklung und Durchführung von Unittests	7		Δ 0	
	Optimierung und Fehlerkorrektur	4		Δ +1	
Abschlussphase	Dokumentation	5	Σ 8	Δ 0	Δ 0
	Übergabe	3		Δ 0	
Gesamt		Σ 80		Δ 0	

Tabelle 3: Reeller Zeitaufwand

Die Analyse des Zeitaufwands zeigt, dass die geplante Gesamtzeit von 80 Stunden exakt eingehalten wurde. Während die Arbeitspakete zur Analyse bestehender Prozesse, Wirtschaftlichkeit, Festlegung

der Projektziele, Implementierung der Schnittstellen, Entwicklung und Durchführung von **Unit-Tests**, Dokumentation und Übergabe exakt nach Plan verliefen, gab es moderate Abweichungen in anderen Bereichen, die sich über den Gesamtprojektzeitraum ausglich:

- **Projektplanung ($\Delta +1h$):** Die leichte Zeitüberschreitung in dieser Phase ist hauptsächlich auf einen etwas erhöhten Aufwand bei der initialen Erfassung der Anforderungen und Spezifikation zurückzuführen. Das präzise Verständnis des bestehenden Systems und die genaue Definition des Umfangs nahmen mehr Zeit in Anspruch als ursprünglich veranschlagt.
- **Designphase ($\Delta +2h$):** In der Designphase benötigte ich jeweils eine zusätzliche Stunde für die Auswahl der Technologien und die Erstellung des technischen Konzepts. Dies ermöglichte eine tiefere **Evaluierung** der Alternativen und eine robustere Architekturplanung.
- **Entwicklungsphase ($\Delta -5h$):** Diese Phase konnte deutlich schneller als geplant abgeschlossen werden. Insbesondere die Umsetzung des Fehlererkennungs- und Meldungssystems sowie das Logging nahmen weniger Zeit in Anspruch als veranschlagt. Dies lag vermutlich an einem effizienten Arbeitsfluss und weniger unvorhergesehenen Problemen bei der Implementierung der Kernlogik.
- **Testphase ($\Delta +2h$):** Die Testphase erforderte zwei Stunden mehr Zeit als ursprünglich geplant. Dies resultierte aus einem erhöhten Aufwand für manuelle Tests mit verschiedenen Datensätzen und der Optimierung und Fehlerkorrektur, da während der Tests unerwartet weitere, wenn auch kleinere, Fehler aufgedeckt wurden, deren Behebung zusätzliche Zeit benötigte.

Die Fähigkeit, auf diese Abweichungen flexibel zu reagieren und den Zeitplan innerhalb der Phasen anzupassen, ohne den Gesamtzeitrahmen zu überschreiten, demonstriert eine effektive Projektsteuerung und realistische Selbsteinschätzung im Projektverlauf.

6.3.2 Zielanalyse

Neben dem Vergleich des Zeitaufwands ist die Bewertung der Zielerreichung ein zentraler Bestandteil des Projektabschlusses. In diesem Abschnitt analysiere ich, inwieweit die im Abschnitt 2.3 definierten Projektziele im Rahmen der betrieblichen Projektarbeit erreicht wurden.

Ziel 1: Bereitstellung eines automatisierten Validierungslaufs.

Dieses Ziel habe ich vollständig erreicht. Ich habe ein eigenständiges **Validierungssystem** entwickelt und erfolgreich implementiert, welches die im Abschnitt “3.1 Ablauf der Anwendung” beschriebene Funktionalität für die automatisierte Ausführung der Validierungsläufe aufweist. Die technische Grundlage für eine regelmäßige Ausführung ist somit geschaffen.

Ziel 2: Implementierung umfassender Validierungsregeln.

Dieses Ziel wurde ebenfalls vollständig erreicht. Ich habe die im Abschnitt “3.3. Bewertung zu Prüferfelder” spezifizierten Validierungsregeln für alle dort genannten Entitätsfelder, einschließlich der Prüfung von Pflichtfeldern, Formaten und Abhängigkeiten, technisch implementiert. Die Logik zur Überprüfung der Datenqualität anhand dieser Kriterien ist im Modul `validation` umgesetzt und einsatzbereit.

Ziel 3: Etablierung eines effektiven Fehlerbenachrichtigungssystems:

Auch dieses Ziel habe ich vollständig erreicht. Ich habe einen automatisierten Benachrichtigungsmechanismus über das Modul `mailer` implementiert, der in der Lage ist, strukturierte Fehlerberichte per E-Mail an die verantwortlichen Teams zu senden. Die Testphase hat gezeigt, dass die E-Mail-Benachrichtigung wie geplant funktioniert und zeitnah erfolgt.

Ziel 4: Sicherstellung der Systemfunktionalität und -stabilität durch Tests:

Dieses Ziel habe ich weitestgehend erreicht. Ich habe umfassende **Unit-Tests** und manuelle Tests für das Validierungssystem konzipiert und durchgeführt. Diese Tests haben dazu beigetragen, die korrekte Funktion der Validierungsregeln und des Benachrichtigungsmechanismus sicherzustellen und kritische Fehler vor der Produktivsetzung zu identifizieren und zu beheben. Eine Testabdeckung der Kernlogik wurde angestrebt und zu einem hohen Grad erreicht, auch wenn eine absolute 100%-Abdeckung in der Praxis oft schwer realisierbar ist und für die erfolgreiche Projektabnahme nicht zwingend erforderlich war. Wichtig ist, dass die zentralen Funktionen zuverlässig getestet wurden.

Die im Abschnitt 2.3 definierten Projektziele wurden somit im Rahmen der vorgegebenen Zeit und des Projektumfangs erfolgreich erreicht. Das entwickelte **Validierungssystem** stellt die geplanten Funktionalitäten bereit und bildet die Grundlage für die angestrebte Verbesserung der Datenqualität und Reduzierung des Prüfaufwands im betrieblichen Alltag. Ziele wie die tatsächliche Erreichung einer spezifischen Datenqualität sind als langfristige, operative Ziele zu verstehen, die durch den Einsatz des entwickelten Systems ermöglicht werden, aber nicht allein durch die Entwicklung im Rahmen dieses Projekts erreicht werden konnten. Mein Projekt liefert das notwendige Werkzeug dafür.

6.4 Fazit

Das Abschlussprüfungsprojekt "Automatisiertes Testen von Entitäten im **Backstage Softwarekatalog**" war für mich eine äußerst wertvolle und lehrreiche Erfahrung. Ich konnte mein während der Ausbildung erworbenes Wissen in einem realen betrieblichen Kontext anwenden und vertiefen.

Die größte Herausforderung lag in der Einarbeitung in die spezifische **Backstage API** und die Gegebenheiten der internen **Backstage**-Instanz, sowie im Umgang mit der paginierten Datenabfrage und optionalen Feldern in den **Entitäten**. Die Implementierung des komplexen Fehlerhandlings und die übersichtliche Aufbereitung der Informationen für die E-Mail-Benachrichtigung stellten ebenfalls anspruchsvolle Aufgaben dar.

Ich bin sehr zufrieden mit dem erzielten Ergebnis. Das entwickelte **Validierungssystem** erfüllt die definierten Anforderungen und leistet einen wichtigen Beitrag zur Verbesserung der Datenqualität im **Backstage Softwarekatalog**. Die wirtschaftliche Betrachtung hat gezeigt, dass das Projekt eine lohnende Investition für das IT-Systemhaus ist, da es zu erheblichen Zeiteinsparungen für die Entwicklungsteams führt.

Besonders positiv empfand ich die Möglichkeit, eigenständig an einem relevanten betrieblichen Problem zu arbeiten und dabei den gesamten Projektlebenszyklus von der Planung bis zur Übergabe zu durchlaufen.

Das Projekt hat meine Fähigkeiten in den Bereichen Anforderungsanalyse, Softwarearchitektur, Implementierung, Testen und Dokumentation deutlich gestärkt. Ich bin zuversichtlich, dass ich die im Projekt erworbenen Kenntnisse und Erfahrungen in meiner zukünftigen beruflichen Laufbahn erfolgreich einsetzen kann.

6.5 Ausblick

Das entwickelte Validierungssystem bietet eine solide Grundlage, die in Zukunft erweitert und optimiert werden kann. Mögliche Weiterentwicklungen könnten umfassen:

- **Integration einer Benutzeroberfläche:** Wie bereits in der Projektabgrenzung erwähnt, könnte eine grafische Benutzeroberfläche integriert werden, um die Konfiguration neuer Validierungsregeln zu erleichtern und die Validierungsergebnisse übersichtlicher darzustellen.

- **Erweiterung der Validierungsregeln:** Die aktuell implementierten Regeln konzentrieren sich auf die wichtigsten Felder. Zukünftig könnten weitere Regeln für zusätzliche Felder oder komplexere Abhängigkeiten zwischen **Entitäten** hinzugefügt werden.
- **Integration in CI/CD-Pipelines:** Das Validierungssystem könnte in die **CI/CD-Pipelines** integriert werden, um **Entitäten** bereits vor dem Einchecken in das **Versionsverwaltungssystem** zu prüfen und so Fehler frühzeitig zu erkennen.
- **Erweiterte Berichterstellung:** Neben E-Mail-Benachrichtigungen könnten weitere Benachrichtigungskanäle integriert oder detailliertere Berichte erstellt und zentral gespeichert werden.
- **Performance-Optimierung:** Bei einer sehr großen Anzahl von Entitäten könnten weitere Optimierungen der Performance in der Datenabfrage und Validierung notwendig werden.

Diese potenziellen Weiterentwicklungen zeigen das langfristige Potenzial des entwickelten **Validierungssystems** und dessen Beitrag zur kontinuierlichen Verbesserung der Datenqualität im **Backstage Softwarekatalog**.

Verweise Quellen:

¹**Tarifvertrag der BA:** <https://www.arbeitsagentur.de/bakarriere/ba-tarifvertrag>
[Aufgerufen am 14.04.2025]

Tools:

²**Visual-Paradigm:** <https://online.visual-paradigm.com/>
[Aufgerufen am 08.04.2025]

³**Excalidraw:** <https://excalidraw.com/>
[Aufgerufen am 09.04.2025]

Glossar

Abkürzung/ Fachbegriff	Ausgeschrieben	Erläuterung
API-Responses		Die Datenpakete, die von einer API als Antwort auf eine Anfrage zurückgesendet werden. Im Projekt sind dies die JSON-Daten , die von der Backstage API geliefert werden und die Entitäten enthalten.
APIs	Application Programming Interfaces	Eine Schnittstelle, die es verschiedenen Softwareanwendungen ermöglicht, miteinander zu kommunizieren und Daten auszutauschen. In diesem Projekt bezieht sich dies insbesondere auf die Backstage API , über die das Validierungssystem die Entitäten des Backstage Softwarekatalogs abruft.
Assertion-Logik		Code-Bestandteile innerhalb von Unit-Tests oder Test-Frameworks, die überprüfen, ob ein bestimmter Zustand im Programmcode erreicht wurde oder ob ein Ergebnis dem erwarteten Wert entspricht.
BA	Bundesagentur für Arbeit	Die Bundesagentur für Arbeit ist die zentrale Institution für Arbeitsmarktpolitik in Deutschland. Sie ist unter anderem zuständig für die Arbeitsvermittlung, Berufsberatung, Zahlung von Entgeltersatzleistungen und Arbeitsförderungen.
Backstage		Backstage ist ein Open-Source-Tool, das zur Verwaltung und Dokumentation von Softwarekomponenten wie Microservices , APIs oder Bibliotheken eingesetzt wird. Es dient als zentrale Plattform, um einen Überblick über die technische Systemlandschaft zu erhalten und die Zusammenarbeit zwischen Entwicklungsteams zu verbessern. Die Backstage-Instanz wird dabei vom IT-Systemhaus selbst gehostet und an die spezifischen Bedürfnisse der Organisation angepasst.
Backstage Softwarekatalog		Ein zentrales Verzeichnis innerhalb von Backstage , das Informationen über alle Softwarekomponenten einer Organisation als Entitäten verwaltet und dokumentiert. Ihr Validierungssystem prüft die Entitäten in diesem Katalog.
Backstage-Instanz		Eine Backstage-Instanz ist eine spezifische Installation und Konfiguration des Backstage-Tools innerhalb einer Organisation. Sie umfasst die Server, die Software und die Konfigurationseinstellungen, die erforderlich sind, um Backstage in der jeweiligen Umgebung zu betreiben. Da Backstage anpassbar ist, kann jede Instanz unterschiedliche Plugins, Konfigurationen und Integrationen aufweisen.
Bibliotheken		Sammlungen von vorgefertigtem Code und Funktionen, die in eigenen Programmen verwendet werden können. Im Kontext des Projekts sind dies sowohl Standard-Bibliotheken von Go als auch externe Pakete wie testify .
BPU	Betriebspraktischer Unterweiser	Erfahrene IT-Fachkräfte in dem IT-Systemhaus, die Auszubildenden bei operativen Themen, Praktika und Feedbacks unterstützen

Abkürzung/ Fachbegriff	Ausgeschrieben	Erläuterung
Debugging		Der Prozess der Identifizierung, Analyse und Behebung von Fehlern (Bugs) im Programmcode. Ein essenzieller Schritt in der Entwicklungsphase und Testphase .
End-to-End-Tests		Testfälle, die einen vollständigen Prozess oder Workflow im System von Anfang bis Ende durchlaufen, um die Interaktion verschiedener Komponenten und die Gesamtfunktionalität aus Anwendersicht zu überprüfen.
Entitäten (entity)		Im Kontext des Backstage Softwarekatalogs sind Entitäten die Abbildungen von Softwarekomponenten wie Microservices , APIs oder Bibliotheken . Sie werden als strukturierte Metadaten in Form von YAML-Dateien oder JSON-Daten dargestellt.
gehostet		Bezeichnet das Bereitstellen und Betreiben einer Anwendung oder eines Dienstes auf einem Server, sodass er für Nutzer oder andere Systeme zugänglich ist. Sowohl die Backstage-Instanz als auch Ihr Validierungssystem werden innerhalb des IT-Systemhauses gehostet.
Git		Ein verteiltes Versionsverwaltungssystem , das zur Verwaltung von Quellcode und zur Nachverfolgung von Änderungen verwendet wird. Es wurde von Ihnen während der Entwicklungsphase genutzt.
GO		Die von Ihnen gewählte Programmiersprache für die Entwicklungsphase des Validierungssystems , bekannt für ihre Effizienz und gute Unterstützung von Parallelverarbeitung.
HTTP-Requests		Anfragen, die von einem Client an einen Server über das HTTP-Protokoll gesendet werden, um Ressourcen abzurufen oder Daten zu senden. Ihr Validierungssystem nutzt HTTP-Requests , um mit der Backstage API zu kommunizieren.
inkosistent		Daten oder Zustände, die nicht den definierten Regeln, Standards oder Erwartungen entsprechen. Im Projekt bezieht sich dies auf Entitäten im Backstage Softwarekatalog mit Fehlern oder Abweichungen.
Intellij IDEA Ultimate		Eine integrierte Entwicklungsumgebung (IDE), die Sie für die Entwicklungsphase Ihres Go-Projekts genutzt haben.
iterativ		Ein Vorgehensmodell, bei dem ein Prozess oder eine Entwicklung in sich wiederholenden Zyklen durchgeführt wird. Sie haben die Entwicklung und das Testen der Komponenten Ihres Systems iterativ durchgeführt.
Jira		Ein Tool für Aufgabenmanagement und Problemverfolgung (Issue Tracking), das im IT-Systemhaus und von Ihnen im Projekt zur Organisation der Arbeitspakete genutzt wurde.
JSON	JavaScript Object Notation	Ein leichtgewichtiges Datenformat, das für den Austausch von strukturierten Daten verwendet wird. Die Backstage API liefert die Entitäten in diesem Format, die von Ihrem Validierungssystem verarbeitet werden.



Komponenten		Eine Komponente ist ein einzelner, abgrenzbarer Bestandteil eines Softwaresystems. Komponenten können verschiedene Formen annehmen, z. B. Microservices , Bibliotheken oder APIs . Jede Komponente hat eine spezifische Funktion und trägt zur Gesamtfunktionalität des Systems bei.
CI/CD	Continuous Integration / Continuous Deployment	Praktiken zur Automatisierung der Softwareentwicklungsprozesse.
Evaluierung		Der Prozess der systematischen Bewertung oder Beurteilung von etwas (z. B. Ergebnissen, Prozessen, Systemen) anhand spezifischer Kriterien.
Metadaten (metadata)		Daten, die Informationen über andere Daten enthalten. Im Backstage Softwarekatalog sind dies die strukturierten Informationen innerhalb einer Entität (z.B. Name, Owner, Typ), die die Softwarekomponente beschreiben.
Microservices		Microservices ist ein Architekturstil für die Entwicklung von Anwendungen, bei der diese als Sammlung von kleinen, unabhängigen Diensten strukturiert werden. Jeder Microservice führt eine spezifische Funktion aus und kann unabhängig von anderen Diensten entwickelt, bereitgestellt und skaliert werden.
Mock-Objekte		Simulierte Objekte, die in Unit-Tests verwendet werden, um das Verhalten realer Abhängigkeiten (z.B. einer Datenbank oder einer externen API) nachzubilden, ohne die echten Abhängigkeiten nutzen zu müssen. Sie werden verwendet, um Tests isolierter und kontrollierbarer zu machen.
modularen Schichtenarchitektur		Ein Architekturmuster, das die Organisation eines Softwaresystems in logische Schichten oder Module vorsieht, wobei jede Schicht spezifische Aufgaben erfüllt und nur mit angrenzenden Schichten interagiert.
omitempty-Annotationen / omitempty-Tags		Spezielle Markierungen (Tags) in Go Struct -Definitionen, die bei der Serialisierung in JSON-Daten (mit der encoding/json -Bibliothek) bewirken, dass ein Feld ausgelassen wird, wenn es den Nullwert seines Typs hat (z.B. nil für Pointer-Felder , " für Strings).
paginiert		Bezeichnet das Aufteilen einer großen Menge von Daten in kleinere, handhabbare Abschnitte oder "Seiten", die dann einzeln abgerufen werden können. Die Backstage API liefert Entitäten paginiert , was eine spezielle Verarbeitung im System erfordert (Pagination).
Pointern (in Go)		Variablen, die Speicheradressen speichern. In Go Structs werden Pointer-Felder oft verwendet, um optionale Werte oder Strukturen in JSON-Daten darzustellen, da der Wert nil anzeigt, dass das Feld fehlt.
Retry-Logik		ine Implementierung, die vorsieht, dass eine fehlgeschlagene Operation (z.B. ein HTTP-Request) nach einer kurzen Wartezeit automatisch wiederholt wird, um temporäre Probleme zu überbrücken.

SMTP	Simple Mail Transfer Protocol	Ein Standard-Netzwerkprotokoll, das für das Senden und Weiterleiten von E-Mails im Internet verwendet wird.
SMTP-Client	Simple Mail Transfer Protocol Client	Eine Software-Komponente, die das SMTP-Protokoll nutzt, um E-Mails über ein Netzwerk zu senden.
Structs (in Go)		Benutzerdefinierte Datentypen in der Programmiersprache Go, die eine Sammlung von Feldern unterschiedlichen Typs definieren.
TV-BA	Tarifvertrag der BA	Der Tarifvertrag, der die Arbeitsbedingungen und Entlohnung der Mitarbeiter der Bundesagentur für Arbeit regelt.
Unit-Tests		Tests , die kleine, isolierte Einheiten des Programmcodes (z.B. einzelne Funktionen oder Methoden) überprüfen, um sicherzustellen, dass sie korrekt funktionieren und die erwarteten Ausgaben liefern.
Unmarshalling		Der Prozess der Konvertierung von strukturierten Daten (wie JSON-Daten) in Objekte einer Programmiersprache.
Validierung		Validierung ist der Prozess der Überprüfung, ob Daten oder ein System den spezifizierten Anforderungen und Standards entsprechen.
Validierungssystem		Ein Validierungssystem ist ein automatisiertes System, das den Prozess der Validierung durchführt. Es überprüft Daten oder Systeme auf Konformität mit definierten Regeln und gibt bei Abweichungen entsprechende Meldungen aus.
Versionsverwaltung		Ein System oder ein Tool (Git), das Änderungen an Dateien (insbesondere Quellcode) im Laufe der Zeit verfolgt und verwaltet, um die Zusammenarbeit zu erleichtern und frühere Versionen wiederherzustellen.
YAML-Datei	YAML: ("YAML Ain't Markup Language")	Ein Format zur Serialisierung von Daten, das menschenlesbar ist und häufig für Konfigurationsdateien verwendet wird. YAML legt den Fokus auf Lesbarkeit und ist eine Alternative zu JSON oder XML.

Tabelle 4: Glossar